Activity: Group REST API Enhancement and Multi-Client Implementation

IT 106 – Application Development and Emerging Technologies

Personal Activity Journal

Submitted By:

Carl Justine O. Tuazon

BSIT – 3B

April 17, 2026

Tasks Performed

I. Activity Requirement Analysis

During this initial phase of the activity, my role as the group leader revolved around analyzing the requirements and facilitating the overall group workflow. As such, my initial tasks included analyzing the REST API demonstration video; gathering the provided activity files to guide the creation of the project; analyzing the subsequent activity requirements; conceptualizing primary ideas that may be applied to the activity in discussion; and consulting the group for the division of roles that I conceived based on individual expertise.

II. Main Tasks

My main task was to design the backend flow in service.php. I experienced difficulty in deciding how to design the structure to support the user flow and the necessary features. I requested meetings to coordinate and finalize the UI/UX design. Ultimately, this made me decide to organize the API into an action-based flow where requests, such as registration, login, course creation, etc. are handled consistently at one endpoint.

```php
try {
    $conn = connectDatabase();

    // Stop if the database does not exist
    ensureSchemaAvailable($conn);

    $action = trim((string)($request["action"] ?? ""));

    // Route the action to the matching handler.
    switch ($action) {
        case "check_registration":
            handleRegistrationCheck($conn, $request);
            break;
        case "register":
            handleRegister($conn, $request);
            break;
        case "login":
```

Figure 1. Main API Action Router

This is the main request router of the API. It reads the $action value from incoming requests and directs it to the correct backend handler (registration, login etc.).

After completing the logic for processing, I moved on to finalizing the remaining backend operations. At this point, I focused on the features like course viewing, enrollment, etc. Maintaining consistency was difficult since the flow mostly required teacher-only and student-only restrictions. I consulted both with Janice regarding the database structure so that some of the queries I used would match the schema, as well as with Vince and Cayla so that the request fields and returned data would fit the client applications. I organized the features, each having separate backend handlers to implement the validations and role-based checks.

```php
// Return course details, using a teacher or student view depending on the caller..
function handleViewCourse(mysqli $conn, array $request): void
{
    $accountId = requireField($request, "account_id");
    $courseId = requirePositiveInteger($request, "course_id");
    $account = requireAccountByPublicId($conn, $accountId);

    if ($account["role"] === "teacher") {
        $course = requireTeacherCourse($conn, $account["account_id"], $courseId);
        $detailLines = buildTeacherCourseDetailLines($course);
    } else {
        $course = requireStudentCourse($conn, $account["account_id"], $courseId);
        $detailLines = buildStudentCourseDetailLines($course);
    }

    respondSuccess("Course details retrieved successfully.", [
        "role" => $account["role"],
        "detail_lines" => $detailLines
    ]);
```

Figure 2. Course View Handler

This handles course viewing for both teachers and students. It first validates the required fields, identifies the account making the request, and then checks whether the user has proper access to the selected course. If the user is a teacher, it returns the teacher version of the course details and vice versa.

III. Integration and Refining of API requests.

At this point, the client applications were already ready for the integration; I refined their final behaviors for testing. The API was already returning consistent data for actions such as login, etc so both the client applications could already use the same endpoint.

```python
def call_api(payload: dict[str, str]) -> ApiResult:
    # Send every request as JSON so it matches the PHP service contract.
    request_text = json.dumps(payload)
    raw_response = ""

    request = urllib.request.Request(
        API_URL,
        data=request_text.encode("utf-8"),
        headers={"Content-Type": "application/json"},
        method="POST",
    )
```

Figure 3. Python Client Request Sender

This is the core request sender of the Python client. It converts the payload into JSON, sends it to the API as an HTTP POST request, and reads the server's response. It also handles possible HTTP, URL, and JSON parsing errors so the client can still display a proper result even if the server response is invalid or the request fail.

```javascript
// Teachers create courses through the same API used by the CLI client.
const formData = new FormData(event.currentTarget);
const result = await callApi({
    action: "create_course",
    account_id: state.session.account_id,
    course_code: String(formData.get("course_code") || "").trim(),
    course_name: String(formData.get("course_name") || "").trim()
});
```

Figure 4. Course Creation

This handles course creation in the web client. It prevents the default form submission, collects the entered course code and course name, and sends them to the API together with the currently logged-in teacher's account ID. After the request is processed, it displays the returned result in the interface.

IV. Creation of Github Repository and Uploading of Files

To facilitate strict collaboration considering the relatively small scale of the activity, most of the meetings and project execution were done in-person; thus, the repository I created served as the final storage of the project files and not the primary avenue for technical collaboration.

Github repository link: https://github.com/crljstn-tzn/rest_serv.git